Let's denote the red node as R. We track the value of $h_R^{(k)}$ as the layers increase.

Layer 0 Graph 1: $h_R^{(0)} = 1$ Graph 2: $h_R^{(0)} = 1$ Status: Identical.

Layer 1(1-hop Neighborhood) In this layer, the red node sums its own value plus the values of its immediate neighbors.Graph 1: R has 2 neighbors. $h_R^{(1)} = 1 + (1 + 1) = 3$.Graph 2: R has 2 neighbors. $h_R^{(1)} = 1 + (1 + 1) = 3$. Status: Identical. Both nodes "see" a degree of 2.

Layer 2 (2-hop Neighborhood) Now the neighbors' neighbors start to influence the sum.Graph 1: The neighbors of R are connected to other nodes (the chain continues).Graph 2: The neighbors of R are connected to each other or back to R. Status: In almost all standard graph configurations where the 4-hop neighborhood differs, the 2nd layer is the first point where the "branching" or "cycling" of the graph structure manifests in the sum.

Layer 3 (3-hop Neighborhood) By the 3rd layer, the messages have traveled far enough to capture the specific local clustering or path lengths that differentiate the two structures. If the graphs are, for example, a simple line vs. a cycle, or have different branching factors at the 2-hop mark, the sum will diverge here.

To achieve mean aggregation, we define ρ_ϕ as a function that simply extracts the first element of any given sequence (or, alternatively, a function that sums the elements of the sequence).If we define:

$$\rho_\phi([h_{u_1}, h_{u_2}, \ldots, h_{u_{|N(v)|}}]) = h_{u_1}$$

Then, when we sum over all possible permutations Π and divide by the total number of permutations $|\Pi|$, each neighbor $u \in N(v)$ will appear in the first position of the sequence exactly $(|N(v)| - 1)!$ times.The Janossy pooling equation (dropping the MLP) becomes:

$$\frac{1}{|N(v)|!} \sum_{\pi \in \Pi} h_{u_{\pi(1)}} = \frac{(|N(v)| - 1)!}{|N(v)|!} \sum_{u \in N(v)} h_u = \frac{1}{|N(v)|} \sum_{u \in N(v)} h_u$$

This is exactly the formula for mean aggregation.

Janossy pooling is more expressive than mean pooling.

Because Janossy pooling can utilize complex, sequence-dependent functions like LSTMs before averaging, it can capture higher-order interactions and structural dependencies between neighbors that simple mean pooling (which treats all neighbors as independent and equal) inherently smooths over. Since we just proved mean pooling is a special case of Janossy pooling, Janossy pooling must be at least as expressive, and in practice, significantly more so.

To perfectly simulate the Breadth-First Search (BFS) algorithm, the GNN must implement a "once visited, always visited" logic. Specifically, a node should have a value of 1 at step $k+1$ if it was already 1 at step k, OR if any of its neighbors were 1 at step k

Message Function: $M(h_v^k) = h_u$

Aggregation Function: $h_{N(v)}^{k+1} = \max(\{h_u^k, \forall u \in N(v)\})$

Update Function: $h_v^{k+1} = \max(h_v^k, h_{N(v)}^{k+1})$

<u>Initialization</u>: At $k = 0$, the source is 1 and all others are 0.
<u>Step 1</u>: Any neighbor of the source will receive a message of 1. The max aggregation will return 1, and the max update will change that neighbor's state to 1.
<u>Propagation</u>: In each subsequent layer k, the "1" value spreads exactly one hop further away from the source, mimicking the BFS frontier.
<u>Persistence</u>: Because of the max update rule, once a node is reached ($h = 1$), it stays 1 for all future layers.

Attribute: "Likes Wine" Likes Wine (D_L): 50 people. 20 like beer, 30 don't.
$I(D_L) = 1 - ((20/50)^2 + (30/50)^2) = 1 - (0.16 + 0.36) = 0.48$
Doesn't Like Wine (D_R): 100 - 50 = 50 people. Beer: 40 (total) - 20 (likes wine) = 20.No Beer: 60 (total) - 30 (likes wine) = 30.
$I(D_R) = 1 - ((20/50)^2 + (30/50)^2) = 0.48$
Weighted Impurity: $(50/100) \cdot 0.48 + (50/100) \cdot 0.48 = 0.48$
Gain (Wine): $0.48 - 0.48 = \mathbf{0}$

Attribute: "Likes Running" Likes Running (D_L): 30 people. 20 like beer, 10 don't.
$I(D_L) = 1 - ((20/30)^2 + (10/30)^2) = 1 - (4/9 + 1/9) = 1 - 5/9 = 4/9 \approx 0.4444$
Doesn't Like Running (D_R): 100 - 30 = 70 people.Beer: 40 - 20 = 20.No Beer: 60 - 10 = 50.
$I(D_R) = 1 - ((20/70)^2 + (50/70)^2) = 1 - (4/49 + 25/49) = 20/49 \approx 0.4082$
Weighted Impurity: $(30/100) \cdot (4/9) + (70/100) \cdot (20/49) = \frac{12}{90} + \frac{140}{490} = \frac{2}{15} + \frac{2}{7} = \frac{44}{105} \approx 0.4190$
Gain (Running): $0.48 - 0.4190 = \mathbf{0.0610}$ (approx)

Attribute: "Likes Pizza"Likes Pizza (D_L): 80 people. 30 like beer, 50 don't.
$I(D_L) = 1 - ((30/80)^2 + (50/80)^2) = 1 - (9/64 + 25/64) = 30/64 = 0.46875$
Doesn't Like Pizza (D_R): 100 - 80 = 20 people.Beer: 40 - 30 = 10.No Beer: 60 - 50 = 10.
$I(D_R) = 1 - ((10/20)^2 + (10/20)^2) = 1 - 0.5 = 0.5$
Weighted Impurity: $(80/100) \cdot 0.46875 + (20/100) \cdot 0.5 = 0.375 + 0.1 = 0.475$
Gain (Pizza): $0.48 - 0.475 = \mathbf{0.005}$

In a complete tree using all 100 attributes, the tree will have a height of 100. Because the training set contains every possible combination of attribute values (2^{100} examples), each leaf node will represent exactly one unique datapoint. The splitting pattern will follow the sequence of attributes $a_1, a_2, \ldots, a_{100}$, creating a massive, perfectly balanced structure where every path from root to leaf is 100 edges long.

The desired tree should consist of only a single split at the root using attribute a_1, resulting in a tree of height 1. Since a_1 correctly predicts the target y for 99

For any point $x \in S$, let $t(x)$ be its nearest local center in $\hat{S}$ (one of the t_{ij} points). Using the "small fact" provided ($(a+b)^2 \leq 2a^2 + 2b^2$), we can bound the distance from an original point x to a final center in T:

$$d(x,T)^2 \leq 2 \cdot d(x,t(x))^2 + 2 \cdot d(t(x),T)^2$$

If we sum this over all $x \in S$:

$$\sum_{x \in S} d(x,T)^2 \leq 2 \sum_{x \in S} d(x,t(x))^2 + 2 \sum_{x \in S} d(t(x),T)^2$$

Notice that the second term is exactly the weighted cost of the centers $\hat{S}$ on the final set T, because $w(t_{ij}) = |S_{ij}|$.

$$\text{cost}(S,T) \leq 2 \cdot \text{cost}(S,\hat{S}) + 2 \cdot \text{cost}_w(\hat{S},T)$$

To find the total error, we compare our intermediate steps to the global optimum T^*.
Since each T_i is the result of running alg on S_i, if alg is an α-approximation, then $\text{cost}(S_i, T_i) \leq \alpha \cdot \text{cost}(S_i, T^*)$.
Summing over all i:

$$\text{cost}(S, \hat{S}) \leq \alpha \cdot \text{cost}(S, T^*)$$

The final step runs alg on the weighted set $\hat{S}$ to get T. By the same logic:

$$\text{cost}_w(\hat{S}, T) \leq \alpha \cdot \text{cost}_w(\hat{S}, T^*)$$

We need to relate the weighted cost $\text{cost}_w(\hat{S}, T^*)$ back to the original optimal cost. For any $t(x) \in \hat{S}$:

$$d(t(x), T^*)^2 \leq 2 \cdot d(t(x), x)^2 + 2 \cdot d(x, T^*)^2$$

Summing this weighted by the number of points associated with each center:

$$\text{cost}_w(\hat{S}, T^*) \leq 2 \cdot \text{cost}(S, \hat{S}) + 2 \cdot \text{cost}(S, T^*)$$

By substituting the bounds from the previous steps into the equation from exercise 3(a), you will find that the total cost is bounded by a constant factor of the optimal cost:

$$\text{cost}(S, T) \leq O(\alpha^2) \cdot \text{cost}(S, T^*)$$

By the definition of algstr, T is the result of running an α-approximation algorithm on the weighted set $\hat{S}$. Therefore, for any other set of k centers (including the global optimal T^*), the cost of T is at most α times the cost of that set:

$$\text{cost}_w(\hat{S}, T) \leq \alpha \cdot \text{cost}_w(\hat{S}, T^*) \tag{1}$$

We want to bound the weighted cost of the optimal centers T^* relative to the local centers. For any $x \in S_i$, let $t(x) \in T_i$ be its assigned center. Using the fact $(a+b)^2 \leq 2a^2 + 2b^2$:

$$d(t(x), T^*)^2 \leq 2 \cdot d(t(x), x)^2 + 2 \cdot d(x, T^*)^2$$

Summing over all $x \in S$:

$$\sum_{x \in S} d(t(x), T^*)^2 \leq 2 \sum_{x \in S} d(x, t(x))^2 + 2 \sum_{x \in S} d(x, T^*)^2$$

Since $\sum_{x \in S} d(t(x), T^*)^2$ is exactly the weighted cost $\text{cost}_w(\hat{S}, T^*)$, we have:

$$\text{cost}_w(\hat{S}, T^*) \leq 2 \cdot \text{cost}(S, \hat{S}) + 2 \cdot \text{cost}(S, T^*) \tag{2}$$

Recognizing that $\text{cost}(S, \hat{S}) = \sum_{i=1}^{\ell} \text{cost}(S_i, T_i)$, the above equation (2) is proven.

Now we expand the cost of the returned centers T on the original set S:

$$\text{cost}(S, T) = \sum_{x \in S} d(x, T)^2 \leq 2 \sum_{x \in S} d(x, t(x))^2 + 2 \sum_{x \in S} d(t(x), T)^2$$

Substitute Equation (1) into the second term:

$$\text{cost}(S, T) \leq 2 \cdot \text{cost}(S, \hat{S}) + 2 \cdot \text{cost}_w(\hat{S}, T)$$

Now substitute Equation (2) into the result:

$$\text{cost}(S, T) \leq 2 \cdot \text{cost}(S, \hat{S}) + 2\alpha \cdot \text{cost}_w(\hat{S}, T^*)$$

$$\text{cost}(S, T) \leq 2 \cdot \text{cost}(S, \hat{S}) + 2\alpha \Big(2 \cdot \text{cost}(S, \hat{S}) + 2 \cdot \text{cost}(S, T^*)\Big)$$

$$\text{cost}(S, T) \leq (4\alpha + 2) \cdot \text{cost}(S, \hat{S}) + 4\alpha \cdot \text{cost}(S, T^*)$$

Because alg is an α-approximation, for each partition i:

$$\text{cost}(S_i, T_i) \leq \alpha \cdot \text{cost}(S_i, T_i^*) \leq \alpha \cdot \text{cost}(S_i, T^*)$$

Summing over all i gives $\text{cost}(S, \hat{S}) \leq \alpha \cdot \text{cost}(S, T^*)$. Substituting this into our equation from before:

$$\text{cost}(S, T) \leq (4\alpha + 2)(\alpha \cdot \text{cost}(S, T^*)) + 4\alpha \cdot \text{cost}(S, T^*)$$

$$\text{cost}(S, T) \leq (4\alpha^2 + 2\alpha + 4\alpha) \cdot \text{cost}(S, T^*)$$

$$\text{cost}(S, T) \leq (4\alpha^2 + 6\alpha) \cdot \text{cost}(S, T^*)$$

This completes the proof. algstr is indeed a $(4\alpha^2 + 6\alpha)$-approximation.

For a fixed row j and a specific item i, let the counter be $c_{j,h_j(i)}$. The value in this counter consists of the true count $F[i]$ plus the counts of any other items $k \neq i$ that happened to collide with i in that row.Let the error in row j be X_j:

$$X_j = c_{j,h_j(i)} - F[i]$$

From the property provided in the context, we know the expected value of this error is:

$$E[X_j] \leq \frac{\epsilon}{e}(t - F[i])$$

Since $t - F[i] \leq t$ (the total number of items), we can simplify this to:

$$E[X_j] \leq \frac{\epsilon t}{e}$$

We want to find the probability that the error X_j is "large." Specifically, let's see the probability that it exceeds ϵt. By Markov's Inequality ($P(X \geq a) \leq \frac{E[X]}{a}$):

$$Pr[X_j \geq \epsilon t] \leq \frac{E[X_j]}{\epsilon t}$$

Substituting our bound for $E[X_j]$:

$$Pr[X_j \geq \epsilon t] \leq \frac{\epsilon t/e}{\epsilon t} = \frac{1}{e}$$

So, for any single row j, the probability that the error is greater than or equal to ϵt is at most $1/e$. The algorithm defines $\tilde{F}[i]$ as the minimum value across all $d = \lceil \ln(1/\delta) \rceil$ rows.For $\tilde{F}[i]$ to be greater than $F[i] + \epsilon t$, every single row must have an error $X_j \geq \epsilon t$. Since the hash functions are independent, the probability that all d rows fail is the product of the individual failure probabilities:

$$Pr[\tilde{F}[i] \geq F[i] + \epsilon t] = Pr[\forall j : X_j \geq \epsilon t] \leq \left(\frac{1}{e}\right)^d$$

Substitute the value of $d = \ln(1/\delta)$:

$$Pr[\tilde{F}[i] \geq F[i] + \epsilon t] \leq e^{-\ln(1/\delta)} = \delta$$

Finally, using the complement rule ($Pr(A^c) = 1 - Pr(A)$):

$$Pr[\tilde{F}[i] \leq F[i] + \epsilon t] \geq 1 - \delta$$

After implemented the algorithm, we have this plot that gives the relative error vs the exact relative frequency of the items. If you look at your scatter plot, we will notice that as we move from left to

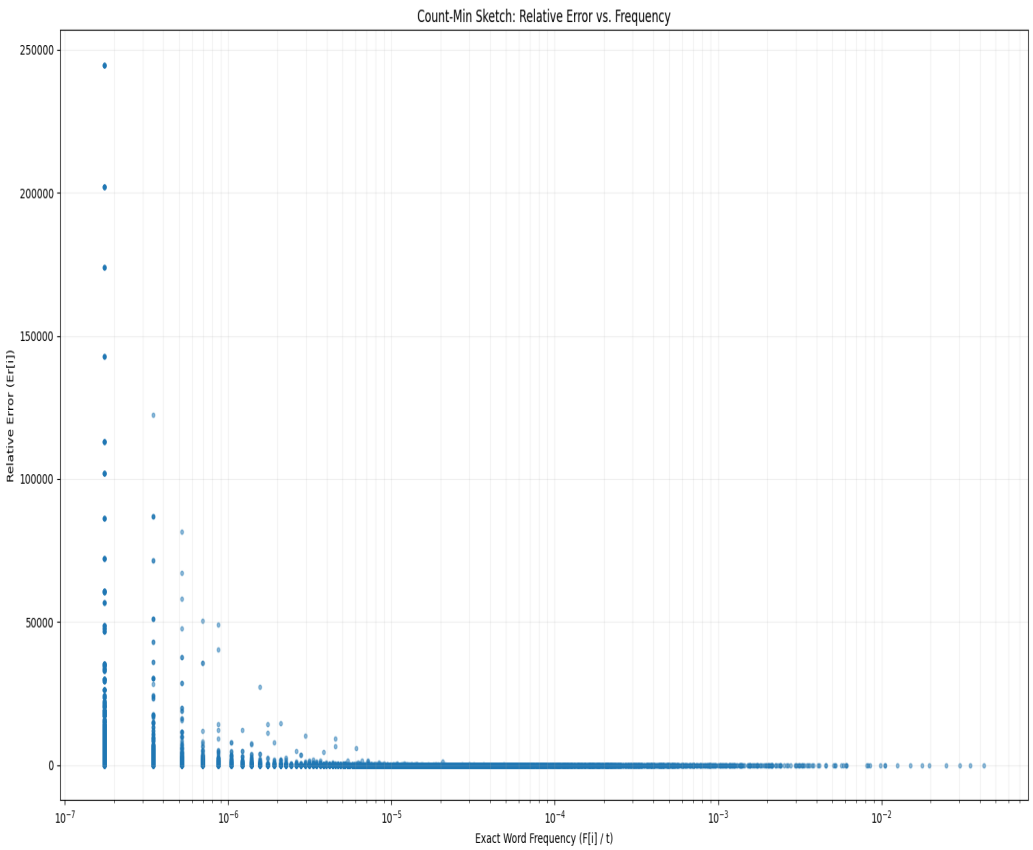

Figure 1: Relative Error vs. Exact Relative Frequency

right along the x-axis, the points "crash" toward the zero-line. The point where the upper cluster of the "cloud" of points consistently drops below the $y = 1$ line will be exactly at $x = \epsilon$.

Information sheet
CS246: Mining Massive Data Sets

Assignment Submission Fill in and include this information sheet with each of your assignments. This page should be the last page of your submission. Assignments are due at 11:59pm and are always due on a Thursday. All students (SCPD and non-SCPD) must submit their homework via Gradescope (http://www.gradescope.com). Students can typeset or scan their homework. Make sure that you answer each (sub-)question on a separate page. That is, one answer per page regardless of the answer length. Students also need to upload their code on Gradescope. Put all the code for a single question into a single file and upload it.

Late Homework Policy Each student will have a total of *two* late periods. *Homework are due on Thursdays at 11:59pm PT and one late period expires on the following Monday at 11:59pm PT*. Only one late period may be used for an assignment. Any homework received after 11:59pm PT on the Monday following the homework due date will receive no credit. Once these late periods are exhausted, any assignments turned in late will receive no credit.

Honor Code We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down their solutions independently, i.e., each student must understand the solution well enough in order to reconstruct it by him/herself. Students should clearly mention the names of all the other students who were part of their discussion group. Using code or solutions obtained from the web (GitHub/Google/previous year's solutions etc.) is considered an honor code violation. We check all the submissions for plagiarism. We take the honor code very seriously and expect students to do the same.

Your name: _____

Email: _____ **SUID:** _____

Discussion Group / AI Usage: _____

I acknowledge and accept the Honor Code.

(Signed) _____